

# YoloV8n model Pothole Detection Working Logic:

## Phase 1: Model Initialization & Optimization

Before processing any video, the AI model needs to be loaded into memory:

```
self.model = YOLO("best.pt")
```

```
self.model.fuse()
```

- `YOLO("best.pt")`: Loads your custom-trained YOLO object detection neural network weights.
- `self.model.fuse()`: Performs a structural optimization on the neural network. It merges the **Convolutional (Conv2d)** layers and **Batch Normalization (BatchNorm2d)** layers into single combined operations. This reduces **mathematical operations during computation, significantly speeding up detection performance (FPS) without losing detection accuracy.**

## Phase 2: Frame Extraction (Video to Still Image)

Videos are simply a sequence of rapidly displayed images (frames). OpenCV is used to open the video file and extract them one by one:

```
self.cap = cv2.VideoCapture(path) # Opens the video stream
```

```
ret, frame = self.cap.read() # Captures a single frame
```

- `self.cap.read()`: Reads the next sequential frame from the video stream.
  - `ret`: A boolean flag (True if the frame was successfully read, False if the video ended or failed).
  - `frame`: A **3-dimensional NumPy array containing raw pixel matrix** data representing an image.
- **Frame format**: OpenCV processes raw image frames in **BGR** (Blue, Green, Red) format with dimensions: (Height, Width, Channels).

### Phase 3: Optimizing Inference Speed (Frame Skipping)

Running an AI model on every single frame can be computationally expensive and slow. To maximize real-time processing speeds, the code implements frame skipping:

```
self.frame_skip = 2
```

```
self.frame_count += 1
```

```
if self.frame_count % self.frame_skip == 0:
```

```
    # Run AI Model Detection
```

- **How it works:** The AI detection pipeline runs **only every 2nd frame**. If `frame_count` is odd, it skips inference completely and continues downstream execution, freeing up processor cycles to keep rendering fluid.

### Phase 4: AI Inference & Pothole Detection

When a frame is passed to the AI model, the following line triggers inference:

```
results = self.model(frame, conf=self.conf_threshold.get(), verbose=False)
```

In short: setting **verbose=False** disables logging output to your console/terminal while the YOLO model processes an image or video frame.

**What happens inside `self.model(frame)`:**

#### 1. Preprocessing:

- OpenCV frames are transformed from standard dimensions into the model's required input matrix size (**e.g.,  $640 \times 640$** ).
- Pixel values (0--255) are normalized to floats (0.0--1.0).

#### 2. Feature Extraction:

- The frame passes through Convolutional layers that identify low-level features (edges, textures) to high-level semantic features (pothole shapes, cracks, roadway context).

#### 3. Bounding Box & Class Prediction:

- YOLO divides the image into a grid and predicts bounding box coordinates along with class probability scores (confidence scores).

#### 4. Filtering by Confidence:

- The `conf=self.conf_threshold.get()` argument discards any detection predictions with a confidence score lower than the slider's value (e.g., < 35% confidence).

## Phase 5: Parsing & Visualizing Results

The results variable returned by the YOLO model contains **structured detection data**:

**results = [**

**Results Object (for current frame) -> [**

**Box 1: {xyxy: [100, 200, 300, 400], conf: [0.89]},**

**Box 2: {xyxy: [500, 600, 700, 800], conf: [0.72]}**

**]**

**]**

`det_count = 0`

`for r in results:`

`for box in r.bboxes:`

`det_count += 1`

`x1, y1, x2, y2 = map(int, box.xyxy[0])`

`conf = float(box.conf[0])`

`# Draw green bounding box around pothole`

`cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)`

`# Draw confidence label above the box`

`label = f"Pothole {conf*100:.1f}%"`

`cv2.putText(frame, label, (x1, y1 - 10),`

`cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)`

### The step-by-step math & drawing:

- `box.xyxy[0]`: Extracts bounding box pixel coordinates in the format `[x_min, y_min, x_max, y_max]`:
  - `(x1, y1)`: Top-left corner coordinate of the detected pothole.
  - `(x2, y2)`: Bottom-right corner coordinate of the detected pothole.
- `box.conf[0]`: Extracts the probability score of the detected pothole (e.g., 0.87 for 87%).
- `cv2.rectangle(...)`: Mutates the image matrix by drawing a green rectangle `((0, 255, 0)` in BGR format) around those exact pixel coordinates.
- `cv2.putText(...)`: Writes text (e.g., "Pothole 87.0%") directly above the bounding box coordinate `(y1 - 10)`.

### Explanation:

#### Understanding YOLO's Data Structure First

When you run `results = self.model(frame)`, YOLO returns a list containing a Results object (one per image passed in). Inside that object sits a `.boxes` container holding **tensor arrays** for every pothole found in that single frame.

Think of results as structured like this:

Plaintext

```
results = [  
    Results Object (for current frame) -> [  
        Box 1: {xyxy: [100, 200, 300, 400], conf: [0.89]},  
        Box 2: {xyxy: [500, 600, 700, 800], conf: [0.72]}  
    ]  
]
```

## Step-by-Step Code Breakdown

### 1. Counter Initialization

Python

```
det_count = 0
```

Resets the detection counter to zero before inspecting the model's predictions for the current frame.

### 2. The Outer Loop: Iterating Over Results

Python

```
for r in results:
```

Iterates through the results list. Because you pass a single image frame to the model, results contains just one Results object r.

### 3. The Inner Loop: Iterating Over Individual Detected Objects

Python

```
for box in r.bboxes:
```

```
    det_count += 1
```

Iterates over every detected object (bounding box) contained inside r.bboxes. If YOLO detected 3 potholes in the frame, this loop runs 3 times, incrementing det\_count each time.

### 4. Extracting Corner Coordinates

Python

```
x1, y1, x2, y2 = map(int, box.xyxy[0])
```

- `box.xyxy[0]`: Extracts the pixel positions of the bounding box as PyTorch tensors formatted as `[x_min, y_min, x_max, y_max]`.
  - `(x1, y1)`: Top-left corner of the pothole.
  - `(x2, y2)`: Bottom-right corner of the pothole.
- `map(int, ...)`: Converts the float values (e.g., 120.45, 210.89) into clean integers (120, 210) because pixel coordinates on an image grid must be whole numbers.

## 5. Extracting Confidence Score

Python

```
conf = float(box.conf[0])
```

Retrieves the model's confidence rating for this prediction as a decimal tensor (e.g., `tensor([0.8742])`) and converts it into a standard Python float 0.8742.

## 6. Drawing the Box Frame

Python

```
cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
```

Directly modifies the image matrix frame by drawing a rectangle:

- `frame`: The target image array.
- `(x1, y1) & (x2, y2)`: Top-left and bottom-right boundary points.
- `(0, 255, 0)`: Color in **BGR** format (0 Blue, 255 Green, 0 Red = Bright Green).
- `2`: Line thickness in pixels.

## 7. Formatting and Writing the Text Label

Python

```
label = f"Pothole {conf*100:.1f}%"
```

```
cv2.putText(frame, label, (x1, y1 - 10),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
```

- `label`: Formats the float 0.8742 into a readable string percentage, e.g., "Pothole 87.4%".
- `cv2.putText(...)`:
  - Draws the text string directly onto frame.
  - Positioned at `(x1, y1 - 10)`, placing the text **10 pixels above** the top-left corner of the bounding box so it doesn't block the visual of the pothole itself.
  - `cv2.FONT_HERSHEY_SIMPLEX`: OpenCV's standard sans-serif font.
  - `0.6`: Text scale size.
  - `(0, 255, 0)`: Green text color.
  - `2`: Font line thickness.

## Phase 6: Image Format Conversion & Loop Recursion

Because OpenCV processes frames in **BGR** format, while Python display tools (like PIL/Tkinter) expect standard **RGB**, a format conversion step is required:

```
# Convert BGR (OpenCV format) to RGB (Standard color format)
```

```
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
# Resize to match window display dimensions
```

```
frame_resized = cv2.resize(frame_rgb, (new_w, new_h))
```

```
# Convert array to PIL Image object
```

```
img = Image.fromarray(frame_resized)
```

### Frame Processing Recursive Loop:

Finally, the system calculates the delay based on real-time FPS and calls the frame processing routine again:

```
delay = max(1, int(1000 / (self.current_fps + 1)))
```

```
self.after_id = self.root.after(delay, self.play_frame)
```

This schedules `self.play_frame()` to fetch the next sequential video frame after delay milliseconds, creating a continuous video loop.

## Concrete Execution Trace Example

Suppose your input video is named road\_test.mp4:

1. **Video Open:** cv2.VideoCapture("road\_test.mp4") opens the file stream.
2. **Frame Capture:** Frame #10 is extracted into a standard pixel array (1080, 1920, 3).
3. **Skipping Check:**  $10 \% 2 == 0$  evaluates to True, so the frame is passed to the AI.
4. **Inference:** YOLO detects a pothole at coordinates (X1=300, Y1=400) to (X2=500, Y2=600) with a confidence score of 0.88.
5. **Drawing:**
  - A green rectangle is rendered spanning pixels (300,400) down to (500,600).
  - The text label "Pothole 88.0%" is rendered at position (300,390).
6. **Color Swap & Scale:** Pixel colors switch from **BGR** to **RGB**, the array is resized down to fit the interface, and the annotated image is displayed.
7. **Next Step:** Frame #11 is loaded immediately afterward[cite: 1]. Since  $11 \% 2 \neq 0$ , AI detection is skipped, saving compute time before Frame #12 undergoes full detection[cite: 1].